

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Программирование»

ОТЧЕТ
к лабораторной работе №9
на тему:
«МНОЖЕСТВЕННОЕ НАСЛЕДОВАНИЕ»
БГУИР 6-05-0612-01 18

Выполнил студент группы 553504
СТЕЦКО Ярослав Юрьевич

(дата, подпись студента)

Проверил ассистент каф. Информатики
РОМАНЮК Максим Валерьевич

(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Задание 1. Вариант 8. Предметная область: Гостиница.

Для заданной предметной области реализовать следующие задачи:

- описать семейство классов, имеющих общий функционал (), при этом в каждом классе присутствует дополнительно свой функционал. Набор дополнительных функций в разных классах может быть произвольным;
- дополнительный функционал описать в виде набора интерфейсов;
- одна из общих функций должна быть реализована по-своему в каждом классе;
- одна из общих функций должна быть реализована в других классах (например, изменение скорости, использование оружия, доставка груза). При этом должно быть несколько вариантов реализации (несколько классов), например, персонажам игры доступны разные инструменты – каждый инструмент может использоваться разными персонажами. Конкретный вариант реализации выбирается при создании объекта (применить шаблон проектирования «Мост» («Bridge»));
- для создания объектов использовать шаблон проектирования «Абстрактная фабрика» или «Построитель» (Builder);
- в классе Program создать коллекцию разных объектов (см. п.1). Затем для каждого элемента коллекции вызвать все методы, доступные для данного объекта.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Перед выполнением работы следует разработать диаграмму классов для наглядного выполнения поставленной задачи (см. рисунок 1).



Рисунок 1 – Диаграмма классов

Для выполнения задания в проект была добавлена папка Classes и Interfaces для удобного взаимодействия с программой (см. рисунок 2).

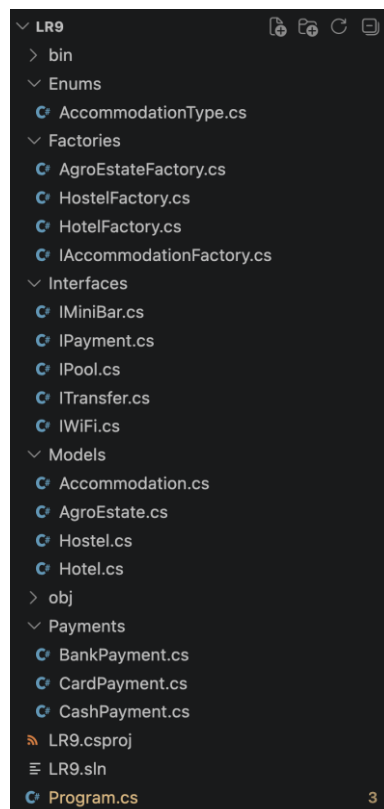


Рисунок 2 – Обзорщик решений

Рассмотрим реализацию интерфейса `IPayment` с методом `Pay()`, который описывает способ оплаты проживания. Классы, реализующие данный интерфейс, будут определены далее.

```
namespace LR9.Interfaces
{
    internal interface IPayment
    {
        string Pay(decimal amount);
    }
}
```

Классы `CashPayment`, `CardPayment` и `BankPayment` реализуют различные способы оплаты: наличными, банковской картой и банковским переводом.

```
using System;
using LR9.Interfaces;

namespace LR9.Payments
{
    public class CashPayment : IPayment
    {
        public string Pay(decimal amount)
        {
            if (amount <= 0)
            {
                throw new Exception("Invalid amount");
            }

            return $"Cash payment: {amount}";
        }
    }
}
```

Рассмотрим реализацию абстрактного класса `Accommodation`, который описывает общие свойства и методы для всех типов жилья. Метод `GetInfo()` выводит информацию об объекте, а метод `Pay()` выполняет оплату через выбранный способ оплаты.

```
using System;
using LR9.Enums;
using LR9.Interfaces;

namespace LR9.Models
{
    public abstract class Accommodation
    {
        protected string name;
        protected AccommodationType type;
        protected IPayment payment;

        public Accommodation(string name,
            AccommodationType type,
```

```

IPayment payment)
{
    if (string.IsNullOrEmpty(name))
    {
        throw new Exception("Name cannot be empty");
    }

    if (payment == null)
    {
        throw new Exception("Payment cannot be null");
    }

    this.name = name;
    this.type = type;
    this.payment = payment;
}

public string Pay(decimal amount)
{
    return payment.Pay(amount);
}

public abstract string GetInfo();
}
}

```

Рассмотрим интерфейсы дополнительных услуг. Интерфейс `IWiFi` описывает использование Wi-Fi, `IPool` отвечает за бассейн, `ITransfer` — за трансфер, а `IMiniBar` — за мини-бар.

```

namespace LR9.Interfaces
{
    internal interface IWiFi
    {
        string UseWiFi();
    }

    internal interface IPool
    {
        string UsePool();
    }

    internal interface ITransfer
    {
        string OrderTransfer();
    }

    internal interface IMiniBar
    {
        string UseMiniBar();
    }
}

```

Рассмотрим наследника класса `Accommodation` — класс `Hotel`. Данный класс реализует интерфейсы `IWiFi`, `IPool`, `ITransfer` и `IMiniBar`, так как отель предоставляет полный набор услуг.

```

using LR9.Enums;
using LR9.Interfaces;

namespace LR9.Models
{
    public class Hotel : Accommodation, IWiFi, IPool, IMiniBar
    {
        public Hotel(string name,
            IPayment payment)
            : base(name, AccommodationType.Hotel, payment)
        {
        }

        public override string GetInfo()
        {
            return $"Hotel: {name}";
        }

        public string UseWiFi()
        {
            return $"{name} uses WiFi";
        }

        public string UsePool()
        {
            return $"{name} uses pool";
        }

        public string UseMiniBar()
        {
            return $"{name} uses minibar";
        }
    }
}

```

Рассмотрим ещё одного наследника класса Accommodation — класс Hostel. Данный класс реализует только интерфейс IWiFi, так как набор услуг в хостеле ограничен.

```

using LR9.Enums;
using LR9.Interfaces;

namespace LR9.Models
{
    public class Hostel : Accommodation, IWiFi, ITransfer
    {
        public Hostel(string name,
            IPayment payment)
            : base(name, AccommodationType.Hostel, payment)
        {
        }

        public override string GetInfo()
        {
            return $"Hostel: {name}";
        }
    }
}

```

```

public string UseWiFi()
{
    return $"{name} uses WiFi";
}

public string OrderTransfer()
{
    return $"{name} ordered transfer";
}
}
}

```

Основная логика проекта реализована с использованием шаблона проектирования **Abstract Factory**. Рассмотрим интерфейс **IAccommodationFactory**, который определяет метод **Create()** для создания объектов размещения.

```

using LR9.Interfaces;
using LR9.Models;

namespace LR9.Factories
{
    public interface IAccommodationFactory
    {
        Accommodation Create(string name,
            IPayment payment);
    }
}

```

Результат работы классов описан в классе **Program**, чтобы наглядно разобраться в структуре созданных классов и интерфейсов.

```

namespace LR9
{
    internal class Program
    {
        static void Main(string[] args)
        {
            List<Accommodation> list =
                new List<Accommodation>();

            IAccommodationFactory factory =
                new HotelFactory();

            IPayment payment =
                new CardPayment();

            Accommodation hotel =
                factory.Create("Radisson", payment);

            list.Add(hotel);

            foreach (Accommodation item in list)
            {
                Console.WriteLine(item.GetInfo());
                Console.WriteLine(item.Pay(100));
            }
        }
    }
}

```

```
    }  
    }  
}
```

Результат работы программы продемонстрирован ниже (см. рисунок 4).

```
Choose accommodation:  
1 - Hotel  
2 - Hostel  
3 - AgroEstate  
0 - Exit  
1  
Enter name: Hilton  
Choose payment:  
1 - Cash  
2 - Card  
3 - Bank  
1  
Object created  
  
Choose accommodation:  
1 - Hotel  
2 - Hostel  
3 - AgroEstate  
0 - Exit  
2  
Enter name: AgroDom  
Choose payment:  
1 - Cash  
2 - Card  
3 - Bank  
2  
Object created  
  
Choose accommodation:  
1 - Hotel  
2 - Hostel  
3 - AgroEstate  
0 - Exit  
0  
All accommodations:  
  
Hotel: Hilton  
Cash payment: 100  
Hilton uses WiFi  
Hilton uses pool  
Hilton uses minibar  
  
Hostel: AgroDom  
Card payment: 100  
AgroDom uses WiFi  
AgroDom ordered transfer
```

Рисунок 4 – Результат работы программы

ВЫВОД

В ходе лабораторной работы были получены навыки проектирования приложения, состоящих из нескольких взаимосвязанных классов и интерфейсов. Создания объектов при помощи шаблона проектирования Builder. Освоены основные навыки разделения абстракции от реализации при помощи шаблона проектирования Bringe.